

The Cost of Centralised Leadership in Distributed Consensus

An Experimental Comparison of Leader-Based and Leaderless Consensus

Sephy Ryu
Independent Researcher
South Korea

Abstract

Leader-based consensus protocols such as Raft concentrate the ordering of replicated writes at a single elected leader, whereas leaderless protocols such as EPaxos distribute ordering across replicas. This paper measures what that difference costs in practice. Both protocols are evaluated under one benchmark methodology: the same workload generator, identical CPU and memory allocations, the same failure injection, and the same metrics, across seven experiment families with ten independent repetitions each and replica counts from three to nine, on a single containerised host. EPaxos sustained a higher mean throughput than Raft at every replica count tested (1.29× to 1.75×, largest at five replicas) and a lower median latency at every replica count; the 95% intervals of the means are separated at three and five replicas but overlap at seven and nine. Throughput decreased for both protocols as replicas were added (57% for EPaxos and 50% for Raft), so the scaling penalty is not specific to leader-based ordering. Aggregate CPU utilisation was similar for the two protocols, but its distribution was not: Raft concentrated utilisation on the current leader, while EPaxos spread it more evenly. Failure behaviour differed by failure role rather than by protocol alone: losing the Raft leader cost seconds of write availability and thousands of failed requests, whereas losing a Raft follower or an EPaxos replica cost tens of milliseconds. The write-ratio and conflict-ratio sweeps showed no systematic trend for either protocol, and EPaxos used about 1.8× the peak memory of Raft in the evaluated implementation.

To separate protocol-level behaviour from implementation choices, each protocol is also evaluated through a second, independent implementation (etcd Raft beside HashiCorp Raft, nvb EPaxos beside the original EPaxos). The four implementations differed sharply in their sensitivity to added one-way communication cost: at 10 ms, throughput fell by 72–96% across the implementations, and the etcd Raft implementation collapsed by 94% at 1 ms already, so absolute performance is a property of the implementation stack, not of the consensus algorithm alone. A separate correctness harness verifies, independently of the performance benchmark, that every implementation performs consistent consensus execution: all 18 deterministic tests (concurrency 1/8/32 plus leader, follower, and replica failures) passed, with no missing or duplicate application and agreement among surviving replicas. The two protocols are represented by different implementations and were measured on a single local host, so these results should not be read as universal properties of either protocol.

Keywords

consensus, Raft, EPaxos, leader-based ordering, leaderless ordering, benchmarking

1 Introduction

Distributed systems often replicate the same state across multiple machines in order to provide fault tolerance and continued service in the presence of failures. When a new operation modifies the state of a replicated state machine, replicas must agree on how that operation is ordered and applied. If different replicas apply operations in different orders, their states can diverge, violating the consistency guarantees expected from the system. Consensus is therefore a fundamental problem in distributed systems.

Paxos is one of the foundational consensus algorithms that established a formal approach to reaching agreement among distributed nodes in the presence of failures [2]. Despite its theoretical importance, Paxos is notoriously difficult to understand and implement. The original presentation is highly abstract and leaves substantial implementation details to system designers. As a result, practical systems based on consensus algorithms may differ considerably in their architecture and implementation choices.

Raft was introduced with a different design goal: making consensus easier to understand and implement while maintaining strong fault-tolerance guarantees [4]. A central feature of Raft is its explicit leader. Client requests that modify replicated state are handled by the leader, which determines the log order and replicates entries to follower replicas before they are committed. In this paper, we refer to this arrangement as *centralised ordering*: the responsibility for determining the order of replicated operations is concentrated in a designated leader, while the other replicas primarily replicate and apply the ordering established by that leader. This structure provides a clear division of responsibilities between the leader and followers and simplifies the coordination required for replicated state machine execution.

The centralised structure of Raft also creates a particular pattern of coordination. Write ordering is concentrated at the leader, and changes in the number of replicas or the offered workload may therefore affect the amount of work performed by the leader and the communication required to commit an operation. Leader failure introduces another distinct condition: because normal request processing depends on the leader, the system must establish a new leader before leader-based processing can resume.

An alternative approach is provided by leaderless consensus. Rather than assigning a single replica the responsibility of

coordinating all operations, leaderless protocols allow multiple replicas to participate directly in determining the ordering and dependencies of operations. EPaxos is a representative example of this approach [3]. It exploits the independence of non-conflicting operations so that replicas can sometimes commit operations without relying on a single global ordering point.

The absence of a central leader, however, does not eliminate the costs of consensus. Leaderless protocols require mechanisms for identifying conflicts, tracking dependencies, and coordinating the execution of operations. These mechanisms can introduce additional communication and computational overhead. The resulting performance therefore depends on the characteristics of the workload and system configuration rather than following directly from the presence or absence of a leader. Factors such as operation conflicts, client concurrency, replica count, read/write composition, and failures can affect the observed behaviour of either approach.

These differences motivate an empirical comparison that separates absolute performance from performance response to changing conditions. A system may achieve higher throughput under one configuration while exhibiting a different rate of degradation as the workload or system size changes. Moreover, measured performance is affected not only by the consensus algorithm itself but also by implementation-level mechanisms such as persistence, scheduling, communication, and resource management. Comparing a single implementation of each protocol can therefore conflate algorithmic structure with implementation-specific behaviour.

In this paper, we experimentally compare Raft and EPaxos as representative leader-based and leaderless consensus protocols. Rather than assuming that either approach is universally superior, we examine how their measured behaviour changes under controlled variations in system scale and workload. The evaluation considers both absolute performance and the sensitivity of throughput and latency to changes in replica count, client concurrency, operation conflict rate, and read/write composition. We additionally examine failure behaviour and the effect of implementation choices on the measured results.

We focus on the following research questions:

- **RQ1:** How do the throughput and latency of leader-based and leaderless consensus respond to changes in replica count, client concurrency, conflict rate, and read/write composition?
- **RQ2:** How does replica failure affect the availability and performance of the two approaches, and how does leader failure in Raft differ from follower failure?
- **RQ3:** To what extent do implementation-level differences affect the measured performance of Raft and EPaxos?

We make the following contributions:

- **Reproducible benchmark infrastructure:** We build the Consensus Lab, which runs Raft and EPaxos in a common execution environment behind a common client harness and interface, providing a controlled basis for comparing their measured behaviour.

- **Sensitivity-oriented evaluation:** We characterise throughput and latency under controlled changes in replica count, client concurrency, conflict rate, and read/write composition, analysing both absolute measurements and their relative changes across configurations.
- **Failure behaviour analysis:** We measure the effects of leader and replica failures on request processing and service availability, distinguishing the effects of Raft leader failure from follower failure and replica failure in EPaxos.
- **Implementation-level analysis:** We examine how different implementations of the same consensus approach affect measured performance, allowing implementation-specific effects to be distinguished from observations that are consistent across implementations.
- **Open infrastructure:** The lab, experiment configurations, and analysis pipeline are released at <https://github.com/Sephy314/Leader-consensus-comparison>, allowing the experiments and analysis to be reproduced and the measured data to be independently re-analysed.

To answer these questions, we conduct controlled experiments comparing Raft and EPaxos under comparable system configurations and workloads. The experiments measure throughput, latency, resource consumption, and behaviour under node failures. Performance sensitivity is analysed using measurements across multiple configurations rather than treating a single benchmark configuration as representative of the protocol as a whole. Because the experiments are conducted in a local environment rather than on physically distributed machines, the results should be interpreted as measurements of protocol behaviour under the specified experimental conditions rather than as universal measurements of performance in production-scale distributed systems.

The remainder of this paper is organised as follows. Section 2 provides background on distributed consensus and reviews leader-based and leaderless approaches, together with related work. Section 3 describes the Consensus Lab, the implementations under test, the client harness and workload model, the metrics, and the experiment suite. Section 4 presents the experimental results on throughput, scalability, latency, resource utilisation, failure behaviour, and implementation differences. Section 5 discusses the results with respect to the research questions and the observed trade-offs between leader-based and leaderless ordering structures. Section 6 discusses threats to the validity of the measurements. Section 7 summarises the findings.

2 Background and Related Work

2.1 Distributed Consensus

Distributed consensus is the problem of enabling multiple distributed processes to agree on a common value or sequence of values despite failures and unreliable communication. In a replicated state machine, this agreement is particularly important because replicas must apply the same sequence of

operations in order to maintain a consistent state. Without an agreement mechanism, different replicas may independently accept or apply different operations, causing their states to diverge.

Consensus therefore provides a fundamental abstraction for building fault-tolerant distributed systems. A consensus protocol must maintain safety, ensuring that correct replicas do not make contradictory decisions, while also providing liveness under the conditions assumed by the protocol. For replicated state machines, these properties are closely connected to the problem of establishing a consistent ordering of state-changing operations.

Consensus protocols can be broadly distinguished by how they coordinate this ordering. One major approach uses a distinguished leader that coordinates the replication of operations, while another distributes the coordination process among multiple replicas without relying on a single permanent ordering authority. Raft represents the former approach, while EPaxos represents the latter.

2.2 Leader-Based Consensus: From Paxos to Raft

Paxos is one of the foundational protocols for distributed consensus. It provides a formal approach to reaching agreement among distributed processes in the presence of failures. However, the original presentation of Paxos is highly abstract, and constructing a complete replicated state machine requires additional mechanisms and implementation decisions beyond the core consensus abstraction [2].

Raft was designed as an alternative consensus algorithm with understandability as a primary design goal. Rather than presenting consensus as a single abstract problem, Raft decomposes it into relatively independent components, including leader election, log replication, and safety [4]. This decomposition provides a more explicit structure for reasoning about how a replicated state machine reaches agreement and maintains correctness.

A central feature of Raft is its leader. At any given term, one replica acts as the leader while the remaining replicas act as followers. Client requests that modify replicated state are normally directed to the leader. The leader appends these operations to its log and replicates the corresponding entries to follower replicas. Once an entry has been replicated to a sufficient quorum, it can be committed and subsequently applied to the replicated state machines [4].

This architecture simplifies the ordering problem by establishing a clear ordering authority. Rather than requiring replicas to independently determine the ordering of every write, followers primarily replicate the log produced by the leader. The Raft design also defines safety properties and mechanisms for preserving them during log replication and changes in leadership [4].

However, the same structure introduces a dependency on the leader. Since the leader is the primary entry point for replicated writes, the write path is structurally concentrated around a single replica. Adding additional follower replicas

therefore does not directly provide additional independent write-ordering capacity. Under sufficiently high write workloads, the leader may become a bottleneck.

The leader also introduces a failure mode that does not exist in the same form in a leaderless protocol. When the current leader becomes unavailable, the remaining replicas must elect a new leader before normal leader-based processing can resume. During this transition, new writes cannot be committed through the normal leader-based path. The election process therefore introduces a period in which failure of the ordering authority can affect request processing.

Thus, Raft provides a clear and structured approach to consensus by centralising the ordering of replicated operations around a leader. This design can simplify coordination and reasoning about correctness, while also creating dependence on the processing capacity and availability of the leader.

2.3 Leaderless Consensus: EPaxos

Leaderless consensus takes a different approach to distributed ordering. Instead of requiring all operations to pass through a single distinguished leader, multiple replicas can participate in proposing operations and determining their dependencies. EPaxos was designed around this approach, motivated in part by the limitations of leader-based consensus when a single replica becomes a central point for coordinating operations [3].

In EPaxos, a client can send a command to any replica, which can act as the replica responsible for proposing that command. This removes the requirement for a single replica to serve as the universal ordering point and allows operations to be proposed concurrently at different replicas [3].

Rather than imposing a single global ordering on all operations through a leader, EPaxos establishes dependencies between commands. Commands that are independent can potentially be committed without requiring them to be ordered relative to one another, while conflicting commands require their dependency relationships to be established consistently [3].

This design allows EPaxos to exploit command independence. When operations do not conflict, replicas can process proposals concurrently instead of serialising all operations through one leader. The resulting degree of parallelism depends on the workload and, in particular, on the extent to which concurrently proposed commands are independent [3].

Leaderless operation also changes the system's behaviour under replica failures. The failure of a particular replica does not necessarily require the remaining replicas to elect a new global leader before they can continue proposing commands. As long as the protocol can obtain the required quorum and satisfy its communication assumptions, other replicas can continue participating in the consensus process [3].

However, removing the single ordering authority does not eliminate coordination. Because multiple replicas can propose commands concurrently, EPaxos must determine the dependencies between commands and ensure that replicas execute conflicting operations in a compatible order. These

mechanisms introduce additional coordination and computation compared with a system in which a single leader directly determines the order of operations [3].

The resulting trade-off is therefore not simply between consistency and availability. Instead, leader-based and leaderless consensus place coordination costs in different parts of the system. Leader-based consensus concentrates ordering around a leader, while leaderless consensus distributes the responsibility for ordering across replicas and relies on dependency management to coordinate concurrent operations.

2.4 Related Work

Prior research has examined both the correctness and practical performance of consensus protocols. Paxos established a foundational framework for distributed agreement, while Raft provided a more explicitly structured leader-based approach with an emphasis on understandability [2, 4]. EPaxos subsequently explored a leaderless approach that exploits command independence and dependency relationships to reduce the reliance on a single ordering authority [3].

The practical evaluation of EPaxos has also been revisited. Tollman et al. re-evaluated EPaxos under geo-replicated workloads and found that the original benchmarking approach did not fully expose the performance impact of conflicting commands. Their results confirmed the low median commit latency reported for EPaxos in WAN settings while observing substantially worse tail latency, with performance being particularly sensitive to application workload [5]. This work illustrates that empirical conclusions about consensus protocols can depend not only on the protocol design itself, but also on workload characteristics and experimental methodology.

Our study follows this experimental perspective while examining a different dimension of consensus behaviour: the effect of centralised versus distributed ordering under a common local experimental environment. We compare a leader-based Raft implementation with an EPaxos implementation while varying replica count, client concurrency, and failure conditions. The comparison therefore evaluates concrete implementations under a controlled workload rather than treating the results as protocol-independent properties of Raft or EPaxos.

The distinction is important when interpreting the measured performance differences. The Raft paper characterises Raft’s strong leadership as a central structural feature of the protocol, whereas EPaxos allows commands to be proposed at different replicas and exploits command independence to avoid relying on a single ordering authority [3, 4]. Our experiments investigate how these architectural differences manifest in throughput, latency, resource utilisation, and recovery behaviour under the evaluated workload.

Accordingly, the results should be understood as evidence about the evaluated protocol implementations and experimental conditions, rather than as universal performance characteristics of all Raft and EPaxos implementations. This distinction is also reflected in the threats to validity discussed in Section 6.

3 Experimental Setup

3.1 Consensus Lab Architecture

The experiments are conducted with the Consensus Lab, a reproducible benchmark infrastructure that runs both protocols in the same execution environment. Each run is a Docker Compose project with one container per role: a coordination service (the upstream EPaxos master [3], or the lab’s raft master which mirrors its RPC surface), one container per replica, and a benchmark client. All containers share a single runtime image and identical CPU and memory allocations.

Both protocols expose the same benchmark-facing interface: the upstream EPaxos `generic smr` wire protocol (`PROPOSE` → `ProposeReplyTS`). A single client binary drives both protocols with the same workload generator, so the op mix, key distribution, and timing are identical. The only protocol-intrinsic difference is endpoint selection: Raft sends every request to the leader reported by HashiCorp Raft, while EPaxos spreads requests round-robin across replicas. Reads are routed through consensus in both protocols; there is no Raft-only local-read optimisation.

3.2 Implementation

Raft is provided by `github.com/hashicorp/raft` v1.7.3 [1], wrapped in a thin adapter (`lab/adapters/raft/`) that converts benchmark requests into state-machine commands, calls `raft.Raft.Apply`, waits for the resulting future, and returns the response. HashiCorp Raft performs leader election, log replication, quorum calculation, commit logic, term management, and state transitions; the adapter never elects or selects a leader. Raft replicas persist state to named volumes (`boltdb`).

EPaxos uses the upstream `efficient/epaxos` server and master unchanged [3] (pinned commit `791b115`, run with `-e -exec -dreply`). The lab only adds external orchestration: containers, monitoring, and failure injection. EPaxos replicas keep state in memory (upstream default). The vendored upstream hardcodes the dependency-set size to 5; the lab extends it to 9 via a documented wire-format extension so that 7- and 9-replica clusters run. The protocol’s phases, quorums, and decision rules are unchanged, and all replicas run the same patched binary.

3.3 Client Harness and Workload Model

The client generates requests with a configurable read/write mix over a key space of 1,000 keys. Write ratio is expressed as the percentage of requests that are writes; the remainder are reads. The client is closed-loop: it runs one worker per unit of configured concurrency, and each worker issues its next request only after the previous request completes, so the configured concurrency is the maximum number of in-flight requests. Both protocols receive the same workload. With probability equal to the configured conflict rate, a request targets one of the shared hot keys; otherwise it targets a uniformly random key from the disjoint cold range, so the

realised hot-key fraction is exact. A warm-up phase precedes the measured phase, and warm-up requests are not recorded.

3.4 Metrics

For every request, the client records the run ID, protocol, replica count, workload ratio, concurrency, operation, key, target replica, start and end timestamps, latency, and success or failure. From these records the processing pipeline computes throughput (requests per second), success rate, and p50/p95/p99 latency distributions.

Per-replica resource samples are collected from the host at approximately 200 ms resolution: CPU from `cgroup v2 cpu.stat`, network RX/TX from `/proc/<pid>/net/dev` in the container’s network namespace, and memory (RSS) from `/proc/<pid>/status`. Raw samples are stored per replica without aggregation, so that workload distribution across replicas can be examined.

3.5 Testbed Configuration and Parameters

All runs use a 20-second measured phase with a 3-second warm-up and ten independent repetitions. Each replica container is allocated 2.0 CPUs and 512 MB of memory; the client gets 2.0 CPUs and 512 MB, and the master 1.0 CPU and 256 MB. `GOMAXPROCS` is set to 4 for every replica in both protocols. The request timeout is 2,000 ms. Raft uses a 1,000 ms heartbeat timeout, a 2,000 ms election timeout, a snapshot threshold of 8,192 entries, and 1,024 trailing logs.

To evaluate the protocols under more realistic network conditions, the benchmark can inject a fixed one-way latency into every inter-replica message, optionally with jitter. The injection is implemented in the adapters’ send path: each outbound message waits a configurable cost plus a uniform random delay before being written to the socket. The cost c is expressed in milliseconds (0, 1, 3, 5, 10); jitter j is a percentage of the cost (0, 5, 10, 50, 100%), so a message at cost c and jitter j waits $c + U(0, c \cdot j/100)$ milliseconds. The baseline (cost 0) is the local-network configuration used by every other experiment family; the jitter sweep is evaluated only at the 5 ms cost level to keep the matrix bounded.

Each repetition is a fresh Docker Compose project: new containers, new networks, and new volumes. Raft’s persisted state (term, voted-for, log, snapshots, cluster membership) lives in per-replica volumes that are destroyed after every repetition and verified removed; persistence is not part of any experiment, so no consensus state survives between repetitions. EPaxos state is in-memory by construction. Before each repetition the harness verifies that the environment is clean (ports free, no leftover containers, volumes, or networks); after each repetition it verifies that teardown removed everything. A repetition that cannot be isolated is recorded as failed, never silently reused.

To prevent experimental conditions from being confounded with wall-clock time, conditions are not executed in blocks. Instead, the runner uses blocked randomization: repetition block b contains every configuration exactly once, in an order shuffled with a seeded random number generator. The

schedule seed is recorded, so the exact execution order is reproducible. Each repetition uses an independent workload seed (base seed plus repetition index), recorded in the run’s metadata, so no two repetitions replay the same workload sequence and any anomalous run can be reproduced exactly.

The experiment dimensions are: replica count (3, 5, 7, 9), write ratio (0, 10, 50, 90, 100%), conflict ratio (0, 10, 25, 50, 75, 90, 100%), client concurrency (1, 2, 4, 8, 16, 32, 64, 128, 256), and one-way communication cost (0, 1, 3, 5, 10 ms) with jitter (0, 5, 10, 50, 100%).

3.6 Experiment Suite

The experiment suite consists of seven families, each with ten repetitions:

- **Workload:** write-ratio (0, 10, 50, 90, 100%) $\times$ concurrency (1–128) sweep at 3 replicas, 2 protocols.
- **Scaling:** replica count sweep (3–9) at 100% writes and concurrency 32, 2 protocols.
- **Conflict:** conflict ratio sweep (0–100%) at 3 replicas, 100% writes, and concurrency 32, 2 protocols.
- **Concurrency:** concurrency sweep (1–256) at 3 replicas and 100% writes, 2 protocols.
- **Communication cost:** one-way latency sweep (0, 1, 3, 5, 10 ms) at 3 replicas, 100% writes, and concurrency 32, followed by a jitter sweep (0, 5, 10, 50, 100%) at the 5 ms cost level, 2 protocols.
- **Election:** Raft leader-election disruption: the leader is killed and the surviving replicas’ Raft transport is isolated for a target number of failed election attempts (0–5). The independent variable is the *measured* number of failed elections (derived from the Raft adapters’ dropped pre-vote and vote counters), not the configured target; the target is a nominal setting only. The measured phase is long enough for the longest isolation to expire and recovery to complete; a run whose cluster does not recover within the phase is recorded as failed.
- **Failure:** Raft leader kill, Raft follower kill, and EPaxos replica kill at 3 replicas, 100% writes, and concurrency 32.

Failure injection is deterministic: the runner kills a container with `docker kill` at a configured offset into the measured phase. For Raft leader failure, the runner queries the raft master for the current leader, maps it to the container hostname, and kills it; the client observes the election and reconnects to the new leader. For EPaxos replica failure, replica 0 is killed and the client moves to the next replica. Recovery timing is reconstructed from request timestamps and the approximately 200 ms monitor resolution, and is therefore approximate.

3.7 Correctness Validation

The experiment families above measure performance, not correctness. To verify that each implementation performs consistent consensus execution, the lab includes a separate correctness harness (`runner correctness`) that reuses the

same Docker Compose infrastructure but runs short, deterministic tests. The harness is independent of the performance benchmark: it never runs in a measured phase, and its observation surface is read-only.

The client’s correctness mode issues a fixed number of requests (2,000) instead of running for a fixed duration. Each request writes its own key (the request id) with a deterministic value (a fixed base plus the request id), so the expected final state is exactly one entry per replied request and a failing test reproduces exactly. Every replica exposes a read-only `GetState` RPC returning a snapshot of its state machine and the number of commands it has applied; the benchmark never calls it. The harness compares replicas on this state, never on their internal log representation, so the EPaxos dependency-ordering model is not assumed to match Raft’s.

Each implementation is tested at concurrency 1, 8, and 32, and under the fault scenarios of the failure family: a Raft follower and a Raft leader are killed and restarted mid-run, and an EPaxos replica is killed and restarted mid-run. Raft replicas persist their log, so a restarted Raft replica must recover the committed state; EPaxos replicas keep state in memory, so a restarted EPaxos replica is only required to be alive.

Host-level telemetry (load average and available memory, sampled at 200ms) is recorded for every run. Host suspension, scheduling gaps, CPU starvation, and clock jumps are detected and recorded per run. Anomalous runs are marked and reported; they are never silently deleted.

The exclusion rule is fixed in advance and applied uniformly to every condition: a run whose host telemetry was flagged contaminated is excluded from every aggregate, and the lost observation is replaced by re-running that repetition, so each condition keeps its full set of independent repetitions. A replacement is a separate run recording which attempt it replaces and why; the original attempt stays in the dataset. Excluded and replacement runs are reported, and the Execution Integrity section of the accompanying report states how many runs were affected.

All aggregates are computed from the raw per-run observations. Each configuration is evaluated over ten independent repetitions; means are reported with sample standard deviation and 95 % confidence intervals (Student’s t). Latency distributions are reported as p50/p95/p99. For the election experiment we report the Pearson correlation between the measured number of failed elections and the availability gap, with n and a t -based p -value, and we plot every observation. Outliers within clean runs are not removed.

4 Evaluation

All reported values are means over ten independent repetitions. Unless stated otherwise, results are for 3 replicas, 100 % writes, and concurrency 32. All figures are generated from the processed benchmark data by `lab/report/figures.py`; each figure shows the mean over repetitions as a line or bar, with the individual repetition values overlaid as points where repetitions are available.

Table 1: Throughput (req/s, mean of 10 repetitions) by replica count, at 100 % writes and concurrency 32.

Replicas	Raft	EPaxos	EPaxos/Raft
3	3,531	5,330	1.51×
5	2,841	4,969	1.75×
7	2,108	2,864	1.36×
9	1,768	2,283	1.29×

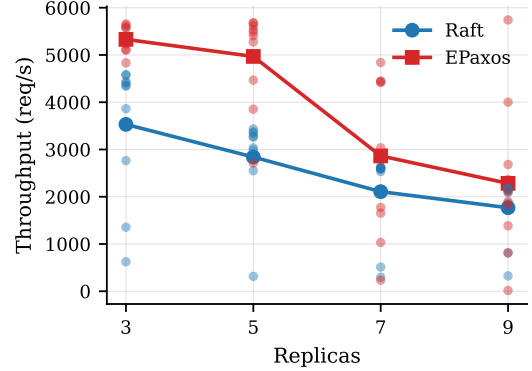


Figure 1: Mean throughput by replica count at 100 % writes and concurrency 32. Points show individual repetitions.

4.1 Throughput and Scalability

Figure 1 shows the measured mean throughput at each tested replica count. Throughput decreased as the replica count increased for both protocols. At the 3-replica baseline (concurrency 32, 100% writes), EPaxos recorded 5,330 req/s and Raft 3,531 req/s. From 5 to 9 replicas, EPaxos fell from 4,969 req/s to 2,283 req/s and Raft from 2,841 req/s to 1,768 req/s (Table 1). Relative to the 3-replica baseline, the 9-replica mean was 57% lower for EPaxos and 50% lower for Raft. The EPaxos/Raft ratio of the means was 1.51× at 3 replicas, 1.75× at 5, 1.36× at 7, and 1.29× at 9.

Because the same nominal configuration (3 replicas, 100 % writes, concurrency 32) is exercised independently by four steady-state experiment families, it also measures the run-to-run variability of a mean over ten repetitions. Those four means were 4,282 (workload), 4,006 (concurrency), 4,221 (conflict at 0 %) and 3,531 req/s (scaling) for Raft, and 4,926, 5,011, 5,147 and 5,330 req/s for EPaxos. EPaxos’s four means lie within 8% of each other; Raft’s span 21%, with the scaling-family mean the lowest and the most variable across repetitions (SD 1,453 req/s against a mean of 3,531). Protocol differences below this magnitude should not be read out of a single family.

Figure 2 shows the measured throughput across the concurrency sweep. Throughput increased with client concurrency for both protocols at every tested level. At concurrency 1,

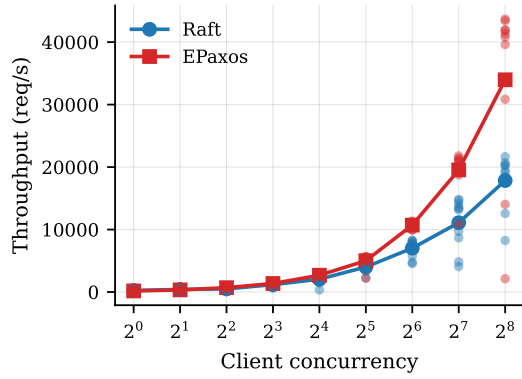


Figure 2: Mean throughput by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

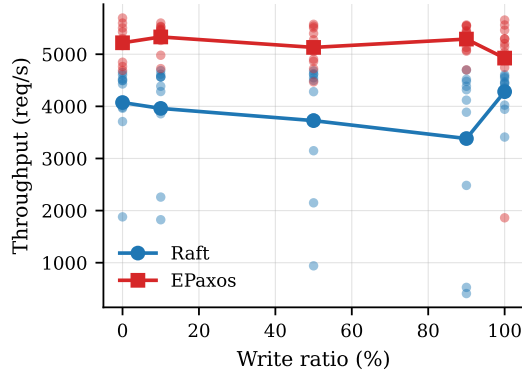


Figure 3: Mean throughput by write ratio at 3 replicas and concurrency 32. Points show individual repetitions.

Raft recorded 257 req/s and EPaxos 174 req/s. At concurrency 256, EPaxos recorded 33,971 req/s and Raft 17,845 req/s. Across the sweep, the highest per-run value observed was 43,681 req/s for EPaxos and 21,656 req/s for Raft, both at concurrency 256. From concurrency 64 to 256, the Raft mean increased by a factor of 2.5 (7,020 to 17,845 req/s) and the EPaxos mean by a factor of 3.2 (10,712 to 33,971 req/s).

Figure 3 shows the measured throughput across the write-ratio sweep at concurrency 32. Between 0 % and 100 % writes, the mean for EPaxos ranged from 4,926 to 5,332 req/s and for Raft from 3,382 to 4,282 req/s. Neither protocol showed a systematic trend across the write ratios tested: Raft’s highest mean was at 100 % writes and its lowest at 90 %, while EPaxos’s highest was at 10 % writes (8% above its 100 % value).

Figure 4 shows the measured throughput across the conflict-ratio sweep. Mean throughput ranged from 3,514 to 4,425 req/s for Raft and from 4,662 to 5,235 req/s for EPaxos across conflict ratios of 0–100 %. The spread across repetitions within

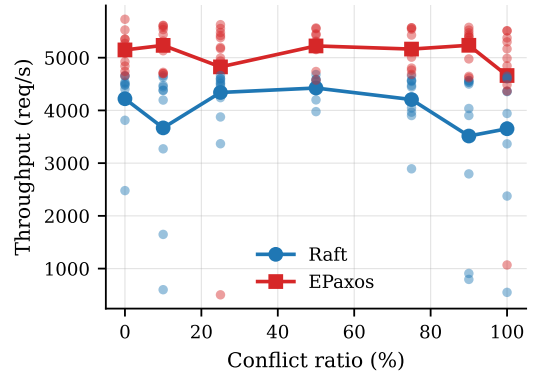


Figure 4: Mean throughput by conflict ratio at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions.

Table 2: Median (p50) latency in milliseconds by replica count, at 100 % writes and concurrency 32.

Replicas	Raft p50 (ms)	EPaxos p50 (ms)
3	12.3	5.6
5	16.8	5.6
7	25.5	6.0
9	22.8	6.2

a configuration was of comparable magnitude to the spread across configurations in both cases.

4.2 Latency

Figure 5 shows the measured median (p50) latency at each tested replica count. At the 3-replica baseline, the median was 12.3 ms for Raft and 5.6 ms for EPaxos. As replicas were added, the Raft median rose to 16.8 ms at 5 replicas and 25.5 ms at 7, then fell to 22.8 ms at 9, while the EPaxos median stayed between 5.6 ms and 6.2 ms across the same range (Table 2).

Figure 6 shows the measured median latency across the concurrency sweep. Median latency increased with concurrency for both protocols. At concurrency 1, the Raft median was 4.1 ms and the EPaxos median 5.5 ms; at concurrency 32, 7.4 ms and 5.9 ms; and at concurrency 256, 13.4 ms and 14.0 ms.

4.3 Resource Utilisation

Figure 7 summarises the per-replica resource measurements. Across all runs, mean per-replica CPU utilisation was 35.8 % for Raft and 35.0 % for EPaxos; mean peak per-replica RSS was 26.4 MB for Raft and 46.8 MB for EPaxos; and mean per-replica network rates were 441 kB/s RX and 443 kB/s TX for Raft, and 408 kB/s RX and 392 kB/s TX for EPaxos.

Figure 8 shows the per-replica CPU utilisation in the 9-replica scaling runs. In the Raft runs, the per-replica means

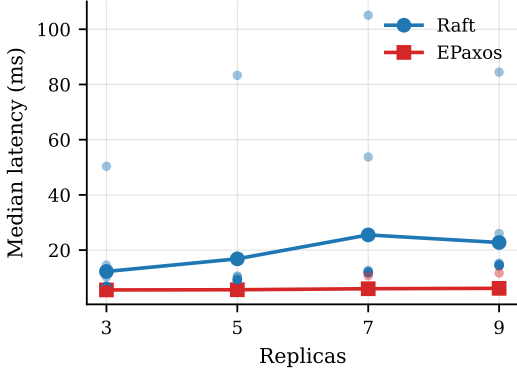


Figure 5: Median (p50) latency by replica count at 100 % writes and concurrency 32. Points show individual repetitions.

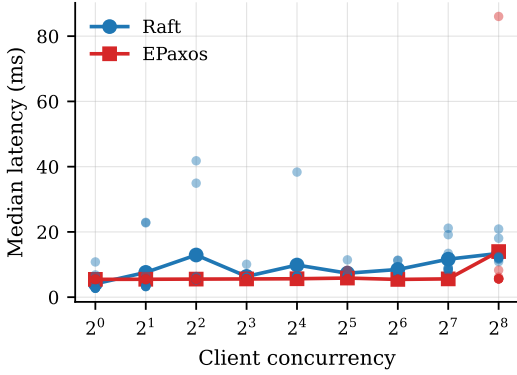


Figure 6: Median (p50) latency by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

ranged from 16.3 % to 29.4 %: whichever replica held the leader role during a repetition used more CPU, and the leadership rotated across repetitions, so each replica’s mean reflects the fraction of repetitions in which it led. In the corresponding EPaxos runs, the per-replica means ranged from 22.4 % to 23.0 %, with no replica standing out.

4.4 Failure Recovery Behaviour

Failure outcomes differed by failure type (Table 3, Figure 9). Killing the Raft leader produced a write-availability gap of 1.81–4.74 s and 2,169–4,133 failed requests. Killing a Raft follower produced a gap of 0.02–0.04 s with no failed requests. Killing an EPaxos replica produced a gap of 0.02–0.09 s with 10–106 failed requests. All runs completed with status `success`; the failed requests are those issued during the transition.

The election family, which isolates replicas to force repeated failed elections during a Raft leader failure, produced

Table 3: Failure behaviour at 3 replicas, 100 % writes, concurrency 32. Gap is the largest interval with no successful write completion after failure injection.

Failure	Protocol	Gap (s)	Failed req.
Leader kill	Raft	1.81–4.74	2,169–4,133
Follower kill	Raft	0.02–0.04	0
Replica kill	EPaxos	0.02–0.09	10–106

availability gaps that grow with the measured number of failed elections (Figure 10, left): across the 60 measured runs the Pearson correlation between the measured failed elections (0–13) and the gap is $r = 0.90$ ($p < 10^{-4}$). This correlation is partly mechanical: the injection isolates the surviving replicas’ transport for a duration proportional to the target, so a run with more failed elections has a longer gap by construction. To separate the injection duration from the recovery behaviour, the right panel shows the recovery time measured from the end of the isolation window to the first successful request. This decoupled metric is smaller and more variable but still increases with the measured election count ($r = 0.41$, $p = 0.001$, $n = 60$): after the isolation expires, a cluster that experienced more failed elections takes longer to restore service, beyond the extra time the injection itself consumed.

4.5 Correctness Validation

The measurements above assume that each implementation performs correct consensus execution. The correctness harness described in Section 3 verifies this independently of the performance benchmark. After each test the harness collects every replica’s state-machine snapshot over the admin RPC and checks that (i) every replied request was applied exactly once, with no missing and no duplicate application; (ii) the surviving replicas’ final states agree; (iii) after a restart, a restarted Raft replica’s state matches the committed state, because Raft persists its log and every replica must catch up, while a restarted EPaxos replica is only required to be alive, because EPaxos keeps its state in memory; and (iv) new requests are processed after the fault.

All 18 tests passed (Table 4): every replied request was applied exactly once on every replica, the surviving replicas agreed on the final state, the restarted Raft replicas recovered the committed state, and requests issued after each fault were processed normally. The per-test results are recorded in `lab/results/correctness/correctness.json`.

4.6 Communication Cost

The communication-cost family adds a fixed one-way latency (cost) to every inter-replica message, optionally with jitter, simulating a real network instead of the local loopback used by the other families. Figure 11 shows the measured mean throughput at each cost level (jitter 0) for the four implementations. The four implementations differ sharply in their sensitivity to added latency. The HashiCorp Raft implementation fell from 4,502 req/s at cost 0 to 1,280 req/s at 10 ms

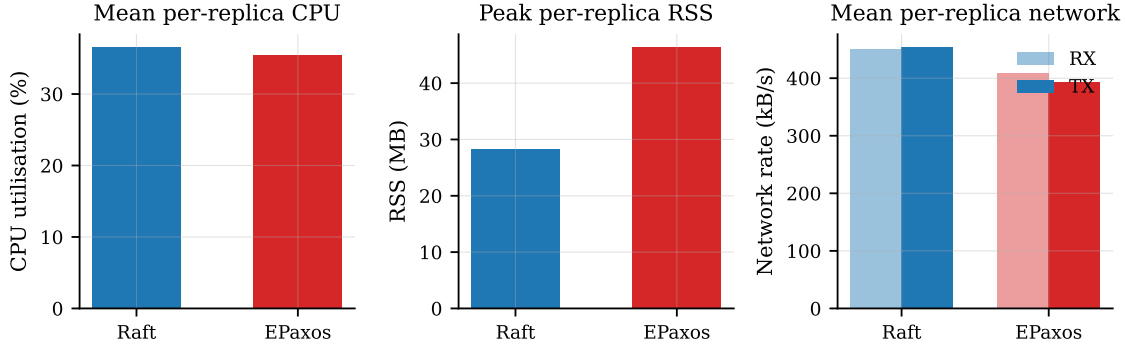


Figure 7: Mean per-replica CPU utilisation, peak per-replica RSS, and mean per-replica network rates, aggregated over all runs.

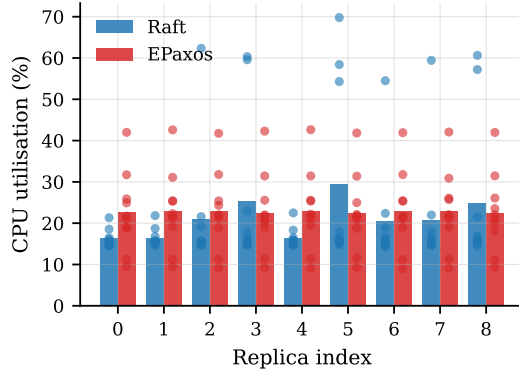


Figure 8: Per-replica CPU utilisation in the 9-replica scaling runs. Bars show the mean over repetitions; points show individual repetitions.

Table 4: Correctness-validation results: 2,000 deterministic requests per test at 3 replicas, 100 % writes. Every replied request was applied exactly once on every replica, surviving replicas agreed on the final state, and requests issued after each fault were processed normally.

Implementation	Sequential	c=8	c=32	Fault
Raft (HashiCorp)	PASS	PASS	PASS	follower/leader PASS
Raft (etcd)	PASS	PASS	PASS	follower/leader PASS
EPaxos (original)	PASS	PASS	PASS	replica PASS
EPaxos (nvb)	PASS	PASS	PASS	replica PASS

(a 72 % reduction), the most gradual decline of the four. The nvb EPaxos implementation fell from 4,346 to 833 req/s (81 %), and the original EPaxos implementation from 5,527 to 221 req/s (96 %). The etcd Raft implementation collapsed at the smallest tested cost: 6,420 req/s at cost 0 fell to 375 req/s at 1 ms (94 %) and to 45 req/s at 10 ms. The 95 % intervals

of the means are separated between the implementations at every cost level above 0.

The median latency grew correspondingly: at 10 ms cost the p50 was 22.6 ms for HashiCorp Raft, 36.2 ms for nvb EPaxos, 125.9 ms for the original EPaxos, and 703.4 ms for etcd Raft (Figure 12). The ordering of the implementations is the same for latency as for throughput, and the etcd Raft p50 at 10 ms cost exceeds the 2,000 ms request timeout only at the largest jitter levels.

Jitter had a much weaker effect than the cost itself. At the 5 ms cost level, raising the jitter from 0 % to 100 % reduced throughput from 2,316 to 1,760 req/s for HashiCorp Raft (24 %), from 1,432 to 1,082 req/s for nvb EPaxos (24 %), from 479 to 305 req/s for the original EPaxos (36 %), and from 87 to 59 req/s for etcd Raft (32 %) (Figure 13). The jitter adds at most $c \cdot j/100$ to each message, so at 100 % jitter the mean one-way delay rises from 5 ms to 7.5 ms; the measured throughput loss is consistent with that modest increase in mean delay rather than with the variance itself.

4.7 Summary of Findings

The measurements support the following observations under the conditions tested:

- At each tested replica count, the measured mean throughput for EPaxos was higher than that for Raft, with the largest relative difference at 5 replicas (1.75 $\times$); the 95 % intervals are separated at 3 and 5 replicas but overlap at 7 and 9.
- Throughput decreased with replica count for both protocols; from 3 to 9 replicas the mean decreased by 57% for EPaxos and 50% for Raft.
- At concurrency 256, the measured mean throughput was 33,971 req/s for EPaxos and 17,845 req/s for Raft; the highest per-run values observed were 43,681 req/s and 21,656 req/s respectively.
- Median latency was lower for EPaxos at every tested replica count, and increased with replica count only for Raft.

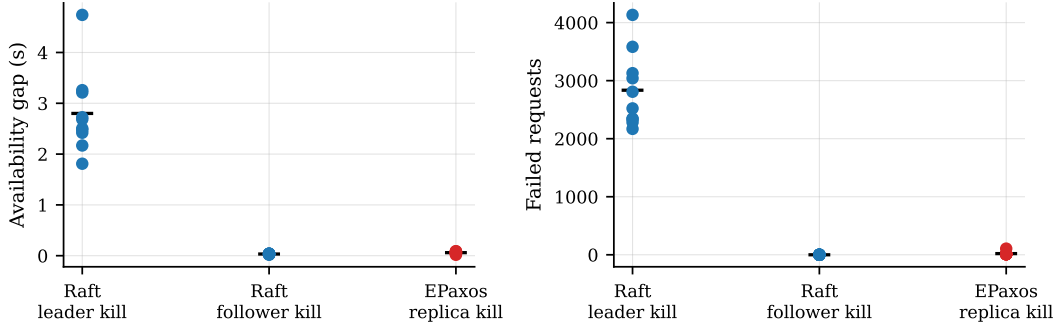


Figure 9: Write-availability gap and failed requests by failure type at 3 replicas, 100 % writes, concurrency 32. Points show individual repetitions; the bar marks the mean.

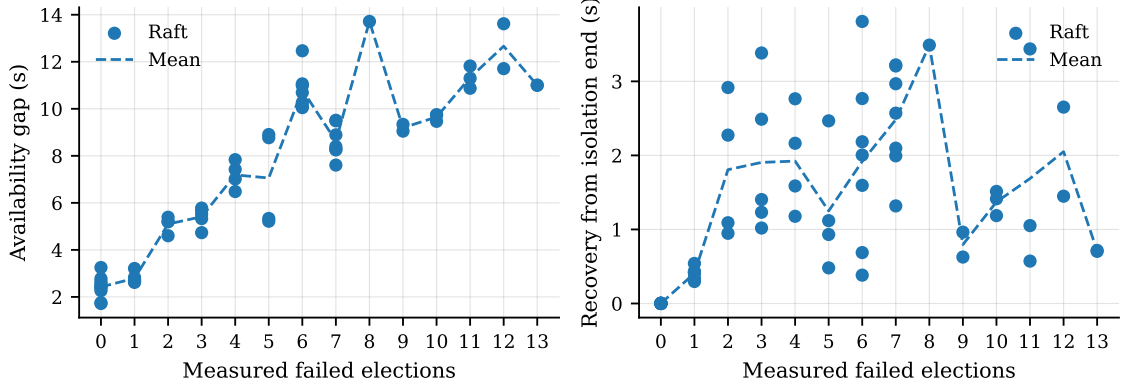


Figure 10: Election-failure recovery in Raft at 3 replicas, 100 % writes, concurrency 32. Left: write-availability gap vs the measured number of failed elections (the gap includes the isolation duration, which is proportional to the target). Right: recovery time from the end of the isolation window to the first successful request, decoupled from the injection duration. Points show individual repetitions; the dashed line marks the mean per value.

- In the 9-replica runs, per-replica CPU means ranged from 16.3 % to 29.4 % in Raft, where leadership rotated between repetitions, and from 22.4 % to 23.0 % in EPaxos, where no replica stood out.
- Write ratio and conflict ratio produced no systematic throughput trend in either protocol over the ranges tested.
- Raft leader-kill experiments recorded write-availability gaps of 1.81–4.74 s and 2,169–4,133 failed requests, while follower-kill and EPaxos replica-kill experiments recorded gaps under 0.1 s with at most 106 failed requests.
- The measured failed-election count (0–13, never the configured target) predicts the availability gap almost linearly ($r = 0.90$, $n = 60$), and remains a significant predictor of the decoupled post-isolation recovery time ($r = 0.41$, $p = 0.001$).

- Under added one-way communication cost, the four implementations differed sharply in sensitivity: HashiCorp Raft lost 72 % of its throughput at 10 ms cost, nvb EPaxos 81 %, the original EPaxos 96 %, and etcd Raft collapsed by 94 % at 1 ms already. Jitter at a 5 ms cost reduced throughput by 24–36 % at 100 % jitter, a much weaker effect than the cost itself.

5 Discussion

5.1 RQ1: Workload and Scaling Sensitivity

5.1.1 Read/Write Composition. The read/write-ratio experiment (Figure 3) shows that throughput does not vary linearly with workload composition. A first-order linear regression did not provide sufficient evidence for a linear relationship, whereas a second-order regression captured the observed curvature. The fitted responses showed a lower-throughput region for mixed read/write workloads and higher throughput towards the read- and write-dominated extremes.

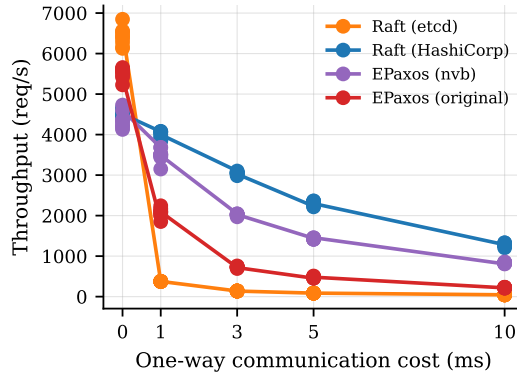


Figure 11: Mean throughput by one-way communication cost at 3 replicas, 100 % writes, and concurrency 32, for the four implementations.

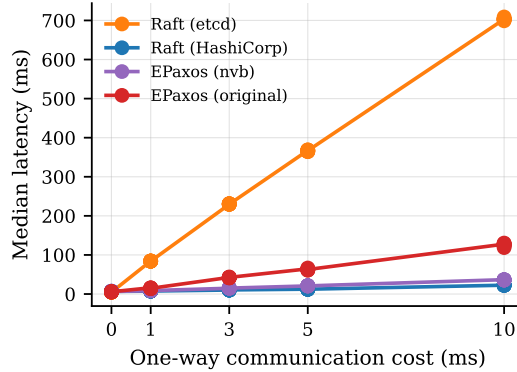


Figure 12: Median (p50) latency by one-way communication cost at 3 replicas, 100 % writes, and concurrency 32, for the four implementations.

The stronger curvature observed for EPaxos indicates that its throughput changed more substantially as the workload moved towards either extreme. This should primarily be interpreted as greater workload sensitivity rather than as evidence of uniformly higher efficiency. The two extremes expose different processing characteristics. In read-dominated workloads, reads do not introduce the same replicated state mutation and log-commit processing associated with writes. In write-dominated workloads, the evaluated EPaxos implementation can distribute proposal processing across replicas, whereas the evaluated Raft configuration directs requests through a single elected leader.

This distinction is particularly relevant to write scalability. In the evaluated Raft configuration, write requests are directed to the elected leader and submitted through the Raft state machine. Consequently, ordering, replication, and associated request-processing work remain concentrated at that replica. Adding replicas therefore does not directly distribute the write-processing path across the cluster. Replica addition

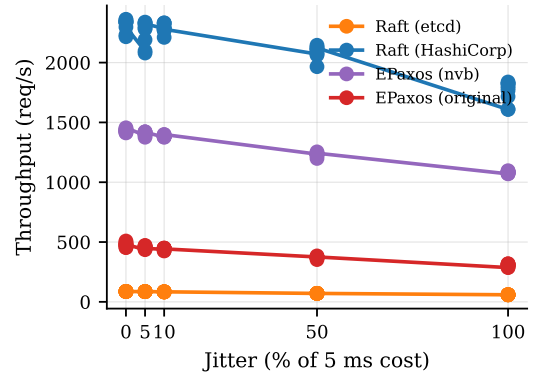


Figure 13: Mean throughput by jitter at a 5 ms one-way cost, 3 replicas, 100 % writes, and concurrency 32, for the four implementations.

can increase the number of available followers for replication and fault tolerance, but it does not by itself create additional leader-side request-processing capacity.

EPaxos has a different processing structure because proposals can be initiated at multiple replicas. Under sufficiently concurrent write workloads, this provides the opportunity to distribute proposal-processing work across the replica set. The higher recovery of EPaxos towards the write-dominated end of the workload sweep is therefore consistent with this architectural difference. However, the curvature alone does not establish a causal relationship between workload composition and a particular internal processing cost; it shows how the measured throughput responds to the composition under the evaluated implementation and environment.

Overall, the read/write experiment demonstrates why a single workload ratio can obscure important differences between consensus implementations. The same protocol may exhibit different relative behaviour depending on whether the workload exposes read processing, replicated writes, or distributed proposal processing.

5.1.2 Replica Scaling. Increasing the replica count reduced throughput for both protocols (Figure 1, Table 1) and increased median latency only for Raft (Figure 5, Table 2). EPaxos nevertheless maintained a higher mean throughput than Raft at every tested replica count, with the relative difference ranging from approximately $1.29\times$ to $1.75\times$. The largest mean difference occurred at five replicas. However, the 95 % confidence intervals of the means overlapped at seven and nine replicas, so the magnitude of the difference at those cluster sizes should not be treated as conclusive.

The reduction in throughput with additional replicas is therefore not specific to leader-based consensus. Both protocols incur additional communication and coordination work as the replica set grows. In EPaxos, larger replica sets can additionally increase the amount of dependency and coordination information associated with proposal processing. The evaluated EPaxos implementation was also modified

to support the dependency-set sizes required for seven- and nine-replica configurations, so results at those sizes include implementation-specific effects in addition to the protocol-level cost of larger clusters.

The Raft results expose a different scaling constraint. Increasing the number of followers increases the amount of replication work associated with a write, but the request-processing path remains centred on the elected leader. Consequently, replica addition does not horizontally distribute the primary write-processing workload. This provides a structural explanation for why adding replicas does not produce additional write throughput in the same way that adding independent request-processing endpoints could.

The scaling experiment therefore separates two effects that can otherwise be conflated. Increasing the replica count provides additional redundancy for both approaches, but it does not necessarily provide proportional additional processing capacity. In particular, the evaluated Raft configuration retains a centralised write path even as the cluster grows.

5.1.3 Client Concurrency. Client concurrency produced a clearer difference in throughput scaling (Figure 2; the corresponding median latency is shown in Figure 6). At low concurrency, the measured throughputs were comparatively close, whereas increasing concurrency produced a substantially larger throughput increase for EPaxos. At the highest tested concurrency, the difference between the protocols was considerably larger than at low concurrency.

This behaviour is consistent with the distribution of request-processing work. In the evaluated Raft configuration, requests are directed to the elected leader. Increasing client concurrency therefore increases the amount of work presented to the same request-processing endpoint. EPaxos, in contrast, permits proposals to be initiated at multiple replicas, allowing additional concurrency to expose processing capacity distributed across the replica set.

The result suggests that centralised request processing becomes more visible as offered concurrency increases. The experiment does not establish that EPaxos universally scales better with concurrency, because the measured response also depends on implementation and workload characteristics. Rather, under the evaluated configuration, the leader-centred request path imposed a stronger throughput constraint at high concurrency.

The concurrency result also complements the read/write experiment. The stronger throughput recovery of EPaxos towards write-dominated workloads is consistent with distributed proposal processing, while the concurrency sweep identifies a workload condition under which that structural difference becomes more apparent.

5.1.4 Conflict Rate. The conflict-ratio experiment (Figure 4) did not reveal a clear monotonic relationship between conflict ratio and throughput. Intermediate measurements varied non-monotonically for both protocols, and no simple linear trend described the complete set of observations. Therefore, the results do not support interpreting every incremental

increase in conflict as producing a corresponding throughput reduction.

A clearer difference appeared when comparing the extreme conflict conditions. Both protocols showed a throughput reduction between the lowest- and highest-conflict workloads, with the reduction being more pronounced for EPaxos. This indicates sensitivity to highly conflicting workloads without establishing a deterministic throughput function of conflict ratio.

The reduction observed for EPaxos at high conflict is consistent with the additional coordination required when its dependency relationships become more complex and commands are less likely to remain independent. In particular, the observation is compatible with the slow-path behaviour of EPaxos. However, the present measurements do not directly instrument the internal execution path sufficiently to establish that mechanism as the cause of the measured reduction. Additional instrumentation under controlled network and scheduling conditions would be required for such a causal analysis.

The conflict results therefore provide stronger evidence for sensitivity at the extremes than for a smooth relationship across the entire conflict range. This distinction is important because local scheduling and container-level variability can obscure relatively small changes between neighbouring conflict levels.

5.2 RQ2: Failure Behaviour

The failure experiments (Table 3, Figure 9) demonstrate a distinct difference between leader failure and non-leader replica failure. Killing the Raft leader produced a substantial availability gap, whereas killing a Raft follower caused little or no observable interruption in the measured workload. EPaxos replica failures also produced only short interruptions compared with the Raft leader-failure case. The Raft leader-kill experiments recorded availability gaps of 1.81–4.74s and 2,169–4,133 failed requests, while follower-kill and EPaxos replica-kill experiments recorded gaps below 0.1s with at most 106 failed requests.

The Raft result follows directly from the request path used in the experiment. The benchmark sends requests to the elected leader, so terminating that replica removes the endpoint through which requests are currently processed. The surviving replicas must subsequently complete leader detection and election before normal request processing can resume. The observed interruption therefore depends not only on the consensus protocol but also on election timing, failure detection, and client reconnection.

The election experiment (Figure 10) further shows that the disruption is related to the difficulty of completing leader election. The measured number of failed election attempts was strongly associated with the observed availability gap ($r = 0.90$, $n = 60$). The association remained significant for recovery time measured after the isolation window had ended ($r = 0.41$, $p = 0.001$), indicating that election difficulty was

not merely a consequence of the duration of the injected isolation period.

The follower-failure result provides an important contrast. When a Raft follower is removed, the elected leader remains available and the remaining replicas retain the quorum required by the evaluated three-replica configuration. The normal request path can therefore continue without the leader replacement step observed after leader failure.

EPaxos does not rely on a single leader endpoint for normal proposal placement. Consequently, failure of one replica does not require election of a replacement leader before other replicas can continue processing proposals. The short interruption observed after an EPaxos replica failure is consistent with this distributed proposal-processing structure.

These observations do not imply that leaderless consensus eliminates failure costs. A failed replica can still reduce available resources, affect quorum conditions, and introduce additional coordination. Rather, the experiment isolates a particular failure-mode difference: losing the replica responsible for the normal request-processing path has a substantially different availability effect from losing a non-leader replica in the evaluated systems.

The failure results therefore add a dimension that steady-state throughput cannot capture. A centralised leader can simplify request routing and provide a well-defined ordering point, but the same concentration creates a specific failure dependency. Conversely, distributing proposal processing removes that single request-processing endpoint, while retaining other costs associated with replica failure and coordination.

5.3 RQ3: Implementation Sensitivity

The implementation-sensitivity experiments show that measured consensus performance depends substantially on the implementation surrounding the protocol. Changes to persistence behaviour, request-processing paths, adapter-level processing, and communication conditions produced performance changes that were large relative to some of the baseline differences between the evaluated protocol implementations.

The clearest example was the Raft implementation. The initial implementation incurred substantial overhead from synchronous persistence in the request path. Removing this persistence bottleneck produced a large throughput increase, while allowing more asynchronous request processing produced further improvements and substantially reduced latency in the evaluated configurations. These changes altered measured performance without changing the underlying Raft consensus algorithm.

This observation is important for interpreting absolute throughput comparisons. A benchmark does not measure an abstract consensus algorithm in isolation; it measures a complete implementation stack consisting of consensus processing, state-machine execution, persistence, networking, serialisation, scheduling, and adapter behaviour. Consequently, a lower throughput measurement for one implementation cannot by itself be attributed to a fundamental limitation of the underlying consensus algorithm.

The communication-cost experiment (Figure 11) reinforces this point. The four evaluated implementations exhibited markedly different sensitivity to injected one-way latency. At 10 ms of one-way cost, HashiCorp Raft lost approximately 72 % of its throughput, nvb EPaxos approximately 81 %, and the original EPaxos implementation approximately 96 %. The etcd Raft implementation showed an even sharper reduction, losing approximately 94 % of its throughput at only 1 ms of injected cost. The corresponding latency measurements also diverged substantially across implementations (Figure 12).

These differences cannot be explained solely by the distinction between leader-based and leaderless ordering, because implementations of the same protocol family exhibited substantially different responses to the same injected network condition. The result instead demonstrates that communication behaviour is mediated by implementation choices such as batching, message scheduling, transport handling, persistence, and request processing.

Jitter produced a comparatively smaller effect than the fixed communication cost in the tested configuration (Figure 13). At a 5 ms one-way cost, increasing jitter to 100 % reduced throughput by approximately 24–36 % across the evaluated implementations. This suggests that the fixed communication delay was a more dominant factor than the tested variation in delay at that operating point. However, this observation should be interpreted only within the tested range and environment.

The implementation experiments therefore motivate a distinction between two levels of comparison. Absolute throughput and latency describe the behaviour of the particular implementation and environment being measured. Relative changes across controlled workload, concurrency, failure, and communication conditions provide evidence about how that implementation responds to changing conditions. The latter can remain informative even when absolute performance changes substantially between implementations.

At the same time, implementation sensitivity does not make protocol-level structure irrelevant. The workload and failure experiments show behaviour consistent with structural differences between the evaluated approaches: centralised request processing and leader dependence in the Raft configuration, versus distributed proposal placement and the absence of a single leader endpoint in EPaxos. The results indicate that these structural properties are observed through an implementation layer that can amplify, reduce, or obscure their performance effects.

Overall, the experiments support a cautious interpretation of consensus benchmarks. Absolute performance should be treated as a property of the evaluated implementation and experimental environment, while claims about protocol behaviour should rely on changes observed under controlled variations. This distinction is especially important when comparing implementations with different persistence, networking, scheduling, and execution architectures.

5.4 Overall Interpretation

Taken together, the three research questions show that the cost of centralised leadership is not represented by a single throughput difference. It appears in several interacting dimensions of the evaluated system.

First, the leader-centred request path limits how additional replicas can contribute to write-processing capacity in the evaluated Raft configuration. Replica addition increases the number of replicas participating in replication but does not move the primary request-processing path away from the leader. This constraint becomes more visible as client concurrency increases and is also reflected in the stronger recovery of EPaxos towards write-dominated workloads.

Second, leadership introduces a specific failure dependency. The Raft leader is both an ordering point and the endpoint used by the benchmark for normal request processing. Its failure therefore interrupts the request path and requires a new leader before normal processing can resume. The corresponding follower-failure experiment shows that this interruption is not an inherent consequence of losing any replica; it is specifically associated with losing the leader in the evaluated architecture.

Third, the magnitude of these effects is implementation-dependent. Persistence, request scheduling, adapter behaviour, and communication handling can substantially change the measured throughput and latency without changing the underlying consensus algorithm. The observed results should therefore not be interpreted as establishing a universal performance ordering between Raft and EPaxos.

The combined evidence instead supports a more specific conclusion: under the evaluated workloads and implementations, distributing proposal processing can reduce the dependence of request processing on a single replica, particularly under high concurrency and write-heavy workloads, while centralised leadership retains a simpler request-processing structure at the cost of concentrating part of the processing and failure path at the leader. The practical significance of this trade-off depends on workload composition, cluster size, communication conditions, failure requirements, and implementation design.

6 Threats to Validity

6.1 External Validity

All replicas run on a single host inside containers over a bridge network, so the measurements contain no wide-area latency, no real packet loss, and no multi-machine failure domains. The host is a WSL2 environment, which places a virtualisation layer between the containers and the hardware. Cluster sizes are 3–9, the key space is 1,000 keys, commands are uniform in size, and the measured phase is 20 s. The results therefore describe short, uniform, single-host runs rather than long-lived deployments with skewed access patterns or large payloads.

The benchmark also places multiple replicas, the client, and the control processes on the same host. Consequently,

processes may compete for host CPU resources, and the measured resource utilisation can depend on host-level scheduling and contention. The experiments detect anomalous resource conditions, but do not provide strict CPU or hardware isolation. Resource utilisation should therefore be interpreted as an observation under the evaluated environment rather than as an implementation-independent resource requirement.

Reads are routed through consensus in both protocols and no Raft-specific read optimisation is enabled, so the read comparison is one of equivalent consistency rather than of each system’s fastest read path; a deployment using leader leases or `ReadIndex` would show a different read profile. Both systems are measured with identical CPU and memory allocations, which is the fair choice for a controlled comparison but is not a tuned deployment of either.

The comparison also uses one implementation of each protocol, so the results should not be interpreted as representative of all Raft or EPaxos implementations. The measured behaviour may depend on implementation-specific choices such as batching, storage, networking, scheduling, configuration defaults, and optimisation level. In particular, the use of HashiCorp Raft and the original EPaxos research implementation means that the measured differences cannot be attributed exclusively to the underlying ordering architecture.

The EPaxos implementation is additionally modified to support the dependency-set sizes required by the 7- and 9-replica configurations. These modifications are part of the evaluated implementation and may contribute to the observed performance at those replica counts. The results at larger cluster sizes should therefore not be interpreted as isolating the cost of the EPaxos protocol from the cost of this implementation extension.

Repeating the comparison with multiple independent implementations of each protocol would provide stronger evidence for generalising from these specific implementations to the broader leader-based and leaderless design classes.

The failure and leader-election experiments are conducted with three replicas. Their recovery times therefore characterise the evaluated three-replica configuration and should not be interpreted as establishing recovery behaviour for larger clusters.

For Raft, the observed leader-recovery interval also depends on the configured heartbeat and election-timeout parameters. The reported recovery times should therefore be interpreted relative to the evaluated timing configuration rather than as protocol-independent bounds on Raft recovery.

6.2 Statistical Conclusion Validity

Each configuration is measured ten times, and aggregates are reported as means with sample standard deviation and a t -based 95 % interval. Ten repetitions leave wide intervals for a variable system: measuring the same nominal configuration in four steady-state experiment families produced means that differed by up to 21 % for Raft and 8 % for EPaxos, and intervals overlap in several comparisons whose means differ, for example at 7 and 9 replicas. We therefore read only

differences that exceed this reproducibility band and treat the overlapping comparisons as inconclusive rather than as small effects. Numerous configurations are compared without a multiplicity correction, so an isolated configuration-level difference should not be read as an established effect.

The 20s measurement window also limits the extent to which the experiments characterise long-term steady-state behaviour. The reported results should therefore be understood as measurements over the specified workload and measurement interval rather than as estimates of indefinite long-running performance. Outliers within clean runs are not removed. Runs identified as affected by environmental interference or measurement failure are investigated separately and are not excluded solely because their performance is inconvenient for the comparison.

7 Conclusion

This paper presented an experimental comparison of Raft and EPaxos as representative leader-based and leaderless consensus approaches. Rather than treating a single throughput measurement as representative of a consensus protocol, the evaluation examined how throughput, latency, scaling behaviour, failure recovery, and resource use changed under controlled variations in workload and system conditions.

The results show that the cost of centralised leadership is most visible when the workload places increasing pressure on the request-processing path. In the evaluated Raft configuration, write ordering, replication, and request processing remain centred on the elected leader. Adding replicas therefore does not directly distribute the primary write-processing path, and increasing client concurrency exposes this concentration more strongly. EPaxos, in contrast, permits proposals to be initiated across multiple replicas, which is consistent with its stronger throughput response under high concurrency and its recovery towards write-dominated workloads. These observations describe the behaviour of the evaluated implementations rather than establishing a universal performance advantage for either protocol.

The read/write-ratio experiment further showed that workload composition has a non-linear relationship with throughput. The stronger curvature observed for EPaxos indicates greater sensitivity to movement between mixed and extreme workload compositions. The conflict experiment, however, did not produce a simple monotonic relationship between conflict ratio and throughput, showing that workload effects cannot always be reduced to a single linear trend.

Failure behaviour revealed another consequence of centralised leadership. Raft leader failure caused a substantial availability gap because the normal request-processing endpoint was lost and a new leader had to be elected. In contrast, Raft follower failure caused little interruption in the evaluated three-replica configuration, while EPaxos replica failure produced only short interruptions. The election experiments further showed that the measured number of failed election attempts was strongly associated with the observed availability gap. These results demonstrate that steady-state throughput

alone does not capture the operational cost of a consensus design.

The implementation-sensitivity experiments provide an important qualification to all of these comparisons. Changes to persistence, request processing, adapter behaviour, and communication conditions substantially affected measured performance without changing the underlying consensus algorithm. Implementations of the same protocol family also showed markedly different sensitivity to injected communication latency. Consequently, absolute throughput and latency should be understood as properties of a particular implementation and experimental environment rather than as intrinsic constants of Raft or EPaxos.

Overall, the experiments indicate that evaluating consensus systems requires considering more than peak steady-state throughput. The distribution of request-processing work, the ability to use additional replicas, workload sensitivity, failure dependencies, and implementation-level costs all contribute to the practical behaviour of a consensus system. In the evaluated environment, EPaxos exposed the benefits and costs of distributed proposal processing, while Raft exposed the simplicity and corresponding concentration of a leader-centred request path.

The study is limited by its single-host containerised environment, tested cluster sizes, workload model, and use of one primary implementation of each protocol. The results should therefore be interpreted as controlled measurements of the evaluated systems rather than universal claims about production deployments. Nevertheless, the experimental methodology provides a basis for reproducing the comparison and for extending it to additional implementations, larger clusters, multi-host environments, and more realistic network conditions. Such extensions would help further separate protocol-level effects from implementation-specific behaviour.

References

- [1] HashiCorp. 2025. HashiCorp Raft: A Raft Consensus Protocol Implementation in Go. <https://github.com/hashicorp/raft>. Version v1.7.3.
- [2] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [3] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There Is More Consensus in Egalitarian Parliaments. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*. ACM, Farmington, PA, 358–372. doi:10.1145/2517349.2522738
- [4] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [5] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 613–632. <https://www.usenix.org/conference/nsdi21/presentation/tollman>